

Movie Ticket Booking System

1. Project Overview

The Movie Ticket Booking System is a web-based application that allows users to browse movies, view showtimes across theatres, select seats, and book tickets online — much like popular platforms such as BookMyShow. It replaces manual box-office queues with a self-service digital experience while giving theatre administrators tools to manage movies, shows, and seat inventory. As a capstone case study, this project lets trainees apply the complete Java Full Stack skill set — relational data modeling, REST API design, transaction/concurrency handling, and building a responsive, interactive front end — on a scenario with realistic business rules such as seat locking and payment handling.

2. Problem Statement

Traditional ticket counters require customers to queue physically and offer no visibility into seat availability before arrival. Phone- or walk-in-based booking is slow, error-prone, and cannot handle demand spikes for popular shows. Theatres also need a way to manage multiple screens, showtimes, and pricing tiers, and to prevent the same seat from being sold to two customers at once. A unified online platform is needed that lets customers book confidently from anywhere while giving theatre staff accurate, real-time control over inventory and pricing.

3. Objectives

- Design a relational schema covering movies, theatres, screens, shows, seats, bookings, and payments.
- Build secure REST APIs using Spring Boot for browsing, seat selection, booking, and cancellation.
- Implement authentication and role-based access (Customer, Theatre Admin, Super Admin) with Spring Security and JWT.
- Prevent double booking through seat-locking during checkout (short-lived lock with timeout and release).
- Integrate a payment flow (real gateway sandbox or a simulated payment service) with booking confirmation.
- Develop a responsive front end with movie listings, an interactive seat map, and a booking history page.
- Generate a downloadable/e-mailed e-ticket with a QR code for entry verification.
- Provide theatre admins with a dashboard for show management and sales/occupancy reporting.

4. Scope of the Project

The system is scoped to a single booking platform serving multiple theatres and screens within one or more cities. It focuses on the core booking lifecycle — browse, select seat, pay, confirm, cancel — and administrative show management. Features such as loyalty/rewards programs, dynamic surge pricing, food-and-beverage add-ons, and third-party marketing integrations are treated as optional stretch goals rather than core requirements, keeping the case study focused on demonstrating solid full-stack architecture and correct handling of concurrent bookings.

5. Key Modules

Module	Description
User Management	User registration/login, profile management, and secure authentication (JWT-based sessions).
Movie & Show Management	Admin/theatre-owner interface to add movies, screens, showtimes, pricing, and language/format (2D/3D) options.
Theatre & Seat Layout	Configurable seat maps per screen (Regular, Premium, Recliner) with real-time seat availability.
Search & Listings	Browse movies by city, language, genre, or release date; view showtimes across theatres.
Seat Selection & Booking	Interactive seat picker with temporary seat-locking to prevent double booking during checkout.
Payment Processing	Integration with a payment gateway (or a mock payment service) to complete transactions securely.
Booking Confirmation & E-Ticket	Auto-generated booking ID, QR-coded e-ticket, and email/SMS confirmation.
Cancellation & Refunds	Ticket cancellation within policy windows with automated refund/seat-release handling.
Admin Dashboard & Reports	Sales reports, occupancy analytics, and show performance dashboards for theatre admins.

6. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, JavaScript, React.js / Angular (Bootstrap/Material UI for responsive design)
Backend	Java, Spring Boot, Spring MVC, Spring Security, REST APIs
Database	MySQL / PostgreSQL, JPA / Hibernate ORM
Concurrency Control	Pessimistic/optimistic locking or Redis-based seat-lock cache to prevent double booking
Build & Version Control	Maven, Git & GitHub
Testing	JUnit, Mockito, Postman (API testing)
Deployment	Apache Tomcat / Docker, deployable on AWS or a local server

7. System Users / Actors

- Customer — searches movies/shows, selects seats, books and cancels tickets, views booking history.
- Theatre Admin — manages screens, showtimes, pricing, and views sales/occupancy reports for their theatre.

- Super Admin — onboards theatres, manages movies catalogue, and oversees platform-wide data.

8. Expected Outcomes / Deliverables

- A fully functional web application with distinct customer and admin views.
- Documented REST API (with a Postman collection of sample requests/responses).
- An entity-relationship diagram and normalized database schema, including the seat-locking design.
- Source code hosted on Git with a clear commit history and README.
- A short demo/presentation covering architecture, concurrency handling, and a live booking walkthrough.

9. Learning Outcomes for Trainees

This case study pushes trainees beyond basic CRUD into concurrency-aware system design — a skill highly valued in real-world backend roles. They will practice designing schemas that model many-to-many relationships (movies-theatres-shows-seats), implementing safe concurrent updates, securing APIs by role, and building a polished, interactive UI for seat selection. They will also gain experience integrating a payment step end-to-end and presenting a technical solution that balances usability with data integrity.

10. Suggested Timeline

- Week 1: Requirement analysis, ER diagram, and database design.
- Week 2: Backend development — entities, repositories, services, and REST APIs (including seat-lock logic).
- Week 3: Security implementation, payment integration, and front-end development.
- Week 4: Integration, testing, bug fixing, deployment, and final presentation.